Midterm 2 — Solutions

Do not distribute before the exam

DATA 412/612

2026-11-11

November 11, 2026. 75 minutes. Questions 1 to 4 are 100 points, 25 each. Question 5 is 10 points of extra credit.

This file matches `midterm-2.qmd`. Accept `|>` or `%>%`. Accept a rebuilt connection when a later question needs `con`. The relative path `data/soccer.sqlite` is what the exam asks for. The path from the notes, `data/soccer/soccer.sqlite`, is also full credit.

1. The database (25 points)

(a) Connect and list the tables (8 points)

Seven tables remain: `Country`, `League`, `Match`, `Player`, `Player_Attributes`, `Team`, and `Team_Attributes`. Counting `sqlite_sequence` as an eighth soccer table is a 2-point deduction. An absolute path is a 1-point deduction. The connection object has to be created with `dbConnect()` and `SQLite()`.

```
con <- dbConnect(SQLite(), "data/soccer.sqlite")

tables <- dbListTables(con)
tables
```

```
[1] "Country"           "League"            "Match"
[4] "Player"            "Player_Attributes" "Team"
[7] "Team_Attributes"   "sqlite_sequence"
```

```
sum(tables != "sqlite_sequence")
```

```
[1] 7
```

(b) A key (9 points)

`Player` has 11,060 rows and 11,060 distinct ids, so `player_api_id` is a primary key there. `Player_Attributes` has 183,978 rows and the same 11,060 ids, so the id repeats and is not a primary key there. Counting in R after `collect()` is full credit. Swapping the two tables is a 4-point deduction. dbplyr may print a one-time warning that SQL aggregates drop missing values. `n()` and `n_distinct(player_api_id)` do not use that rule. Grade the counts below.

```
tbl(con, "Player") |>
  summarise(n = n(), n_id = n_distinct(player_api_id)) |>
  collect()
```

```
# A tibble: 1 x 2
      n  n_id
  <int> <int>
1 11060 11060
```

```
tbl(con, "Player_Attributes") |>
  summarise(n = n(), n_id = n_distinct(player_api_id)) |>
  collect()
```

```
# A tibble: 1 x 2
       n  n_id
   <int> <int>
1 183978 11060
```

`player_api_id` is a primary key of `Player` because each id appears once. It is not a primary key of `Player_Attributes` because each player has many attribute rows.

(c) One season (8 points)

3,326 matches. The mean of `goal_diff` is about 0.333. Home teams score about a third of a goal more than away teams in this season.

`show_query()` has to appear before `collect()`. Omitting the SQL is a 2-point deduction. The sentence has to say that `collect()` brings the rows into R. Before that, the object is a lazy table and the database has not returned the rows. Naming the lazy table and the data frame is enough.

```r
season_matches <- tbl(con, "Match") |>
  filter(season == "2015/2016") |>
  select(date, home_team_goal, away_team_goal) |>
  mutate(goal_diff = home_team_goal - away_team_goal)

show_query(season_matches)
```

```
<SQL>
SELECT
  `date`,
  `home_team_goal`,
  `away_team_goal`,
  `home_team_goal` - `away_team_goal` AS `goal_diff`
FROM `Match`
WHERE (`season` = '2015/2016')
```

```r
season_matches <- collect(season_matches)
nrow(season_matches)
```

```
[1] 3326
```

```r
mean(season_matches$goal_diff)
```

```
[1] 0.3331329
```

`collect()` runs the query and returns an ordinary data frame. The object before `collect()` is only a description of the query.

2. Player names (25 points)

(a) Simple names (10 points)

9,232 simple names. `str_count()` has to be used for both the uppercase letters and the space. `separate()` on the single space is the split. `str_split_fixed()` or `separate_wider_delim()` is full credit. Filtering after the split, or dropping `player_api_id`, is a 1-point deduction. The six rows are the first six after arranging by family name, then first name.

```r
players <- tbl(con, "Player") |>
  select(player_api_id, player_name) |>
  collect()

simple <- players |>
  filter(
    str_count(player_name, "[A-Z]") == 2,
    str_count(player_name, " ") == 1
  ) |>
  separate(player_name, into = c("first_name", "family_name"), sep = " ")

nrow(simple)
```

```
[1] 9232
```

```r
simple |>
  arrange(family_name, first_name) |>
  slice_head(n = 6) |>
  select(player_api_id, first_name, family_name)
```

```
# A tibble: 6 x 3
  player_api_id first_name family_name
          <int> <chr>      <chr>
1         45891 Gerard     Aafjes
2        555442 Nabil      Aankour
3         45469 Naim       Aarab
4        570431 Rolando    Aarons
5         46392 Dani       Abalo
6        574872 Sunday     Abalo
```

(b) Two ways to keep a player (8 points)

341 players remain. Fifteen of them are named Martinez. The first names, in order, are Alex, Carles, Carlos, Emiliano, Hans, Igor, Inigo, Jackson, Javier, Jorge, Josef, Mario, Nacho, Pablo, and Roman. `str_ends(family_name, "ez")` is the ending. `str_detect(family_name, "ez$")` is full credit. Keeping only one of the two conditions is 4 points if the Martinez list is otherwise built from that smaller set.

```r
kept <- simple |>
  filter(first_name %in% c("Luis", "Kevin") | str_ends(family_name, "ez"))

nrow(kept)
```

```
[1] 341
```

```r
kept |>
  filter(family_name == "Martinez") |>
  arrange(first_name) |>
  select(first_name, family_name)
```

```
# A tibble: 15 x 2
   first_name family_name
   <chr>      <chr>
 1 Alex       Martinez
 2 Carles     Martinez
 3 Carlos     Martinez
 4 Emiliano   Martinez
 5 Hans       Martinez
 6 Igor       Martinez
 7 Inigo      Martinez
 8 Jackson    Martinez
 9 Javier     Martinez
10 Jorge      Martinez
11 Josef      Martinez
12 Mario      Martinez
13 Nacho      Martinez
14 Pablo      Martinez
15 Roman      Martinez
```

(c) Initials (7 points)

The five initials are `M.S.` (117), `M.B.` (100), `M.M.` (94), `A.M.` (89), and `M.P.` (84). The periods are part of the string. Missing both periods is a 1-point deduction. Building the initials from the 341 names in part (b), rather than from all 9,232 simple names, is a 3-point deduction. `slice_max()` is full credit when the tie break is alphabetical.

```r
simple |>
  mutate(
    initials = str_c(
      str_sub(first_name, 1, 1), ".",
      str_sub(family_name, 1, 1), "."
    )
  ) |>
  count(initials) |>
  arrange(desc(n), initials) |>
  slice_head(n = 5)
```

```
# A tibble: 5 x 2
  initials     n
  <chr>    <int>
1 M.S.       117
2 M.B.       100
3 M.M.        94
4 A.M.        89
5 M.P.        84
```

3. Potential and sprint speed (25 points)

(a) The players (8 points)

`named` has 373 rows. Reusing `simple` from question 2 is full credit. The chunk below rebuilds it so this part can be read on its own. `str_starts(first_name, "M")` and `str_length(family_name) >= 8` are the two filters. `unite()` puts the names back. A hand-built `str_c(first_name, " ", family_name)` is full credit.

```r
named <- players |>
  filter(
    str_count(player_name, "[A-Z]") == 2,
    str_count(player_name, " ") == 1
  ) |>
  separate(player_name, into = c("first_name", "family_name"), sep = " ") |>
  filter(str_starts(first_name, "M"), str_length(family_name) >= 8) |>
  unite(player_name, first_name, family_name, sep = " ") |>
  select(player_api_id, player_name)

nrow(named)
```

```
[1] 373
```

(b) One row per player (9 points)

One row per `player_api_id`. The summary covers every player in `Player_Attributes`, not only the 373 names. Restricting the summary to those 373 before the means is full credit, because the join in part (c) drops the rest. Dropping `na.rm = TRUE` is a 2-point deduction: a player with one missing sprint speed would then have a missing standard deviation. `sd()` and `var()` followed by a square root are both fine.

```
attr_sum <- tbl(con, "Player_Attributes") |>
  select(player_api_id, potential, sprint_speed) |>
  group_by(player_api_id) |>
  summarise(
    potential_avg = mean(potential, na.rm = TRUE),
    sd_speed = sd(sprint_speed, na.rm = TRUE),
    .groups = "drop"
  ) |>
  collect()

attr_sum
```

```
# A tibble: 11,060 x 3
   player_api_id potential_avg sd_speed
           <int>         <dbl>    <dbl>
 1          2625          61.1    0.469
 2          2752          70.4   11.4
 3          2768          70.6    6.59
 4          2770          73.5   11.5
 5          2790          75.8    2.17
 6          2796          70.5    4.88
 7          2802          78.9    4.01
 8          2805          77.8    2.30
 9          2857          68.2    8.49
10          2862          74.6    1.59
# i 11,050 more rows
```

(c) Join and plot (8 points)

The left join keeps 373 rows. None of these 373 have a missing mean or a missing standard deviation, so the filter does not drop a point. `inner_join()` happens to give the same 373 rows.

It is full credit here. A join that multiplies rows, because the attributes were not summarised first, is a 4-point deduction.

The smooth is `method = "lm"`, `se = FALSE`, black, and dashed. A loess curve is a 1-point deduction. A gray band is a 1-point deduction. Color mapped to a third variable is a 1-point deduction. `theme_bw()`, the title, and the two axis labels are required.

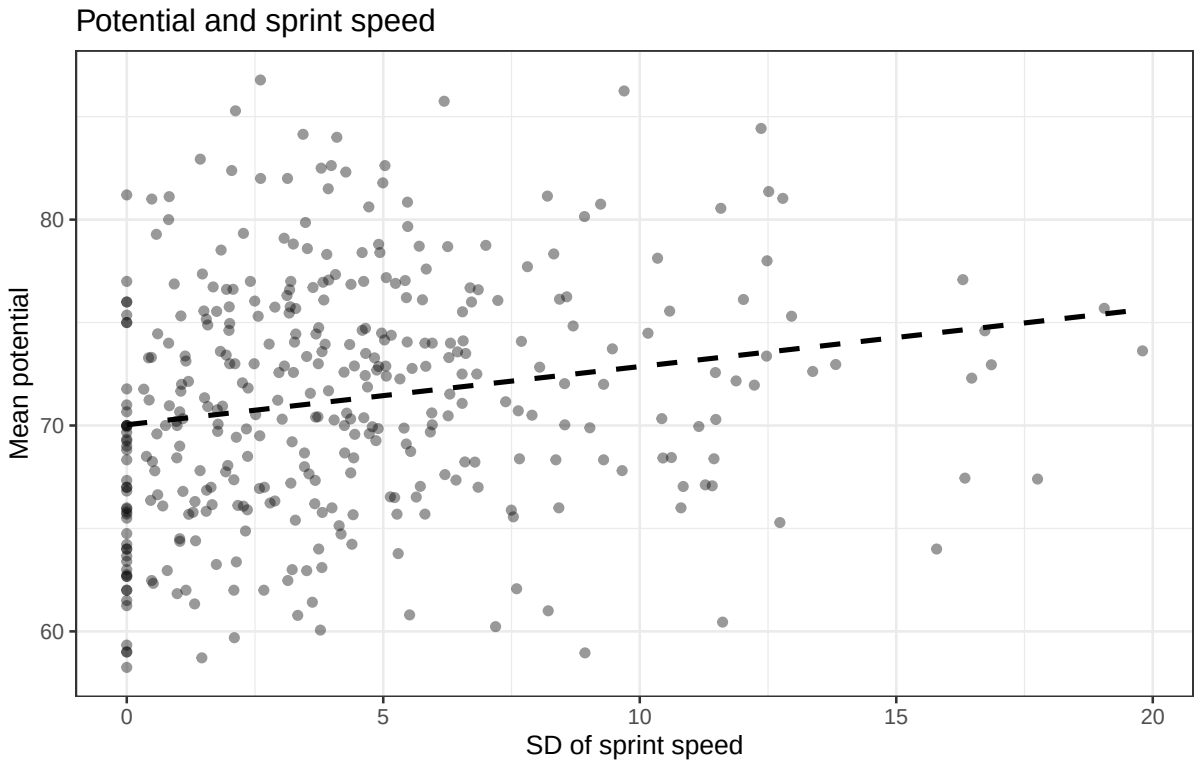
The dashed line rises slightly. Mean potential is only a little higher when the standard deviation of sprint speed is higher, and the cloud is wide. "A weak positive relationship" is full credit. "A strong increase" is not.

```
plot_df <- named |>
  left_join(attr_sum, by = "player_api_id") |>
  filter(!is.na(potential_avg), !is.na(sd_speed))

nrow(plot_df)
```

```
[1] 373
```

```
ggplot(plot_df, aes(x = sd_speed, y = potential_avg)) +
  geom_point(alpha = 0.4) +
  geom_smooth(method = "lm", se = FALSE, color = "black", linetype = "dashed") +
  theme_bw() +
  labs(
    x = "SD of sprint speed",
    y = "Mean potential",
    title = "Potential and sprint speed"
  )
```



4. Team names and birthdays (25 points)

(a) Long names (10 points)

43 long names contain a non-ASCII character. Four names change when the leading number is removed:

- `1. FC Köln` becomes `FC Köln`
- `1. FC Nürnberg` becomes `FC Nürnberg`
- `1. FSV Mainz 05` becomes `FSV Mainz 05`
- `1. FC Kaiserslautern` becomes `FC Kaiserslautern`

`FSV Mainz 05` keeps the `05`. The pattern is anchored at the start, so a number later in the name stays. Removing every digit in the name is a 2-point deduction.

```
teams <- tbl(con, "Team") |>
  select(team_api_id, team_long_name, team_short_name) |>
  collect() |>
  mutate(
    team_name = str_remove(team_long_name, "^\\d+\\.\\s+")
```

```
  )

sum(str_detect(teams$team_long_name, "[^\\x00-\\x7F]"))
```

```
[1] 43
```

```
teams |>
  filter(team_name != team_long_name) |>
  select(team_long_name, team_name)
```

```
# A tibble: 4 x 2
  team_long_name       team_name
  <chr>                <chr>
1 1. FC Köln           FC Köln
2 1. FC Nürnberg       FC Nürnberg
3 1. FSV Mainz 05      FSV Mainz 05
4 1. FC Kaiserslautern FC Kaiserslautern
```

(b) Short names and FC (8 points)

Three short names are not three characters: Rio Ave FC (`RA`), Gil Vicente FC (`GV`), and BSC Young Boys (`YB`).

47 names contain `FC` as a whole word. 50 contain the letters `FC` anywhere. The three extra names are `FCV Dender EH`, `Dijon FCO`, and `GFC Ajaccio`.

`FC` matches those letters inside a longer word. `\\b` requires a break before and after `FC`, so `FCV`, `FCO`, and `GFC` do not count.

```
teams |>
  filter(str_length(team_short_name) != 3) |>
  select(team_name, team_short_name)
```

```
# A tibble: 3 x 2
  team_name      team_short_name
  <chr>          <chr>
1 Rio Ave FC     RA
2 Gil Vicente FC GV
3 BSC Young Boys YB
```

```
sum(str_detect(teams$team_name, "\\bFC\\b"))
```

```
[1] 47
```

```
sum(str_detect(teams$team_name, "FC"))
```

```
[1] 50
```

```
teams |>
  filter(
    str_detect(team_name, "FC"),
    !str_detect(team_name, "\\bFC\\b")
  ) |>
  select(team_name)
```

```
# A tibble: 3 x 1
  team_name
  <chr>
1 FCV Dender EH
2 Dijon FCO
3 GFC Ajaccio
```

(c) The month in the birthday (7 points)

1,073 players have month 02. Eleven of those birthdays are on the 29th. `str_extract()` of the month, plus a second extract of the day, is full credit. `ymd_hms()` and then `month()` and `day()` are full credit on the counts. The pattern has to be applied after `collect()`.

February 29 is a real day in a leap year. The years stored with these eleven birthdays are leap years, so the date is valid.

```
birthdays <- tbl(con, "Player") |>
  select(birthday) |>
  collect()

parts <- str_match(birthdays$birthday, "^(\\d{4})-(\\d{2})-(\\d{2})")

sum(parts[, 3] == "02")
```

```
[1] 1073
```

```
sum(parts[, 3] == "02" & parts[, 4] == "29")
```

```
[1] 11
```

5. A function (10 extra credit)

The name is not graded. `last_word` below is one acceptable name. A pipe inside the function is acceptable. It is not required. The whole question is 10 extra-credit points.

The four results are `Martinez`, `Sadio`, `Luisinho`, and the stop message. The comma row drops `,22` and then takes the last remaining word. `Luisinho` has no space, so it comes back unchanged. The last call has length 2, so it stops. Leaving `NA` untested in the calls is fine. The function still has to reject a missing string.

```
last_word <- function(name) {
  if (!is.character(name) || length(name) != 1 || is.na(name)) {
    stop("name must be one string")
  }
  name <- str_squish(name)
  name <- str_remove(name, ",.*")
  name <- str_squish(name)
  if (!str_detect(name, " ")) {
    return(name)
  }
  str_extract(name, "\\S+$")
}

last_word("  Luis   Martinez ")
```

```
[1] "Martinez"
```

```
last_word("Abdoulaye Diallo Sadio,22")
```

```
[1] "Sadio"
```

```r
last_word("Luisinho")
```

```
[1] "Luisinho"
```

```r
last_word(c("Aaron", "Taylor"))
```

```
Error in `last_word()`:
! name must be one string
```